STINK3014 (NEURAL NETWORKS)
A261 Individual Assignment #1

Instructor: Associate Prof. Dr Azizi Ab Aziz
Submission date: 6th November 2026 (before 11.59pm) via learning portal

"Life is like a box of chocolates. You never know what you're gonna get." [*Forest Gump*]

This individual assignment contains TWO (2) parts. You are required to answer all parts.

PART I: THEORETICAL COMPONENTS

1. Explain the role of activation functions in an MLP. Compare ReLU and sigmoid in terms of their mathematical properties and typical applications.
2. How does the number of hidden layers and neurons affect the learning capacity, computational cost, and risk of overfitting of an MLP?
3. Why is feature scaling important when training an MLP? Explain how differences in the scales of input features can affect the training process.
4. What is backpropagation, and how does it calculate gradients to update the weights and biases of a neural network?
5. Explain the roles of the forward pass and backward pass during MLP training. How do these two processes work together to minimise the loss function?
6. What is the chain rule in calculus, and why is it essential for calculating gradients in a multilayer neural network?
7. What are vanishing and exploding gradients? Explain how they can affect neural network training and identify one method for mitigating each problem.
8. A neural network achieves 98% training accuracy but only 78% validation accuracy. What does this result suggest, and how could hyperparameter tuning help address the problem?

PART II (PRACTICAL TASK): MLP-BASED LOAN APPROVAL PREDICTION USING BACKPROPAGATION

Task Objectives:
By completing this assignment, students should be able to:

- Implement an MLP trained using backpropagation for loan approval prediction.
- Compare SGD, Momentum, AdaGrad, RMSprop, and Adam optimisers.
- Tune network architecture and training hyperparameters systematically.
- Apply L1, L2 or Elastic Net regularisation to control overfitting.
- Implement dropout and assess its effect on generalisation.
- Compare models using appropriate classification metrics and learning curves.
- Deploy the best model for practical application

BACKGROUND

Financial institutions receive loan applications from individuals with different income levels, credit histories, employment circumstances, and asset holdings. Assessing these applications requires identifying patterns in historical data to support consistent lending decisions. In this case, a financial institution wants to use historical loan application data to predict whether a loan application is likely to be approved or rejected. You are required to construct an MLP model and investigate how different hyperparameter selection and regularisation strategies affect its predictive performance. The assignment must answer this central research question:

> "How do optimiser selection, hyperparameter tuning, regularisation, and dropout affect the accuracy, convergence, and generalisation of an MLP for loan approval prediction?"

Dataset Characteristics:

The dataset contains 4,269 records with 13 attributes related to applicant characteristics, financial information, asset values, and a loan decision. The target variable has two classes: Approved (2,2656 records) and Rejected (1,1613 records). The target classes are not perfectly balanced. Students should therefore evaluate precision, recall, F1-score, and ROC-AUC in addition to accuracy. The numerical inputs have different scales. For example, *cibil_score* is in the hundreds, while asset and income values can reach tens of millions. Standardisation is important because gradient-based training can be affected by differences in feature scales.

Students should also consider the conceptual relationship between the variables. A predictive model can learn associations in historical decisions without necessarily establishing that those associations are fair, causal, or appropriate for future lending.

Dataset variables and their roles:

Attribute	Description	Role
loan_id	Unique loan record identifier	Excluded
no_of_dependents	Number of dependents	Input
education	Applicant's education status	Input
self_employed	Self-employment status	Input
income_annum	Annual income	Input
loan_amount	Requested loan amount	Input
loan_term	Loan term	Input
cibil_score	Credit score	Input
residential_assets_value	Residential asset value	Input
commercial_assets_value	Commercial asset value	Input
luxury_assets_value	Luxury asset value	Input
bank_asset_value	Bank asset value	Input
loan_status	Approved or rejected	Target

Task #1: Data preparation and baseline MLP

Using the provided CSV file:

1. Load the dataset using Pandas and inspect its structure.
2. Check for missing values, duplicate records, categorical variables, and class distribution.
3. Exclude loan_id from the input features because it is an identifier rather than a meaningful predictive feature.

4. Encode education and self_employed into numerical values.
5. Encode loan_status as a binary target.
6. Split the data into 70% training, 30% testing sets using stratification.
7. Standardise numerical inputs using training-set statistics only.
8. Construct and train a baseline MLP using backpropagation for **[11- 32 (ReLU)- 16 (ReLU) - 1 (Sigmoid)]** architecture with binary cross-entropy as the loss function, batch size = 32, epochs =500, and Adam as the initial optimiser.

Questions:

1. What is the purpose of forward propagation and backpropagation?
2. How does the network update its weights to reduce prediction error?
3. Why is binary cross-entropy suitable for this task?
4. Why are ReLU and sigmoid suitable for the hidden and output layers, respectively?
5. What evidence in the learning curves suggests overfitting or underfitting?

Task #2: Optimiser Comparison

Train the same baseline architecture with five optimisers:

Optimiser	Initial learning rate	Specific rate for selected optimiser
SGD and Momentum	0.01	$\beta = 0.9$, nesterov=False
AdaGrad	0.001	initial_accumulator_value =0.1, , $\varepsilon = 1e-7$
RMSprop	0.001	$\rho = 0.9$, $\varepsilon = 1e-7$
Adam	0.001	$\beta 1 = 0.9$, $\beta 1 = 0.99$, $\varepsilon = 1e-7$

* Keep the data splits, architecture, loss function, batch size, and maximum number of epochs consistent (as in Task #1).

Compare:

- Training and validation loss
- Validation ROC-AUC
- Test accuracy and F1-score after selecting the model
- Convergence speed and training time

Questions

1. Which optimiser converges most quickly?
2. Which produces the lowest validation loss?
3. How does the learning rate affect convergence?
4. Does the fastest-converging optimiser necessarily produce the best generalisation?

Task #3: Hyperparameter Tuning

Investigate how changing the MLP configuration affects loan approval prediction. Hyperparameters determine how the network is configured and trained. Unlike weights and biases, they are not directly learned through backpropagation.

Hyperparameter	Suggested Values
Hidden layers	1-5
Neurons per layer	8, 16, 32, 64
Learning rate	0.001, 0.01, 0.1, 0.5
Batch size	16, 32, 64
Optimizer	SGD + Momentum, AdaGrad, RMSProp, Adam
Epochs	500-2000

Students must test at least **EIGHT (8)** distinct configurations, using a systematic approach such as random search or a controlled experiment.

Requirements:

- Select the best configuration using a justified evaluation metric.
- Record all tested configurations and their results.
- Avoid selecting hyperparameters using the test set.
- Evaluate the final selected model on the held-out test set only after tuning.
- Identify the best configuration using testing results.

Questions:

1. Which hyperparameters have the greatest effect on performance?
2. Does increasing the number of hidden layers always improve prediction?
3. How does batch size affect training time and model performance?
4. Why should hyperparameters not be selected using test-set performance?

Task #4: Regularisation

Compare the following models using the same architecture and data splits:

- Model A: No regularisation
- Model B: L1 regularisation with λ=0.01
- Model C: L1 regularisation with λ=0.5
- Model D: L2 regularisation with λ=0.01
- Model E: L2 regularisation with λ=0.5
- Model F: Elastic Net regularisation with α= 1.0, λ=0.5

Apply the regularisation penalty to the hidden-layer weights.

Example:

```
from tensorflow.keras import regularizers
from tensorflow.keras.layers import Dense
hidden = Dense( 32,  activation="relu", kernel_regularizer=regularizers.l2(0.001))
```

Questions:

1. How does L1, L2, Elastic Net regularisation modify the loss function?
2. How can penalising large weights reduce overfitting?

3. What happens when the regularisation strength is too high?
4. Which configuration gives the best balance between training and testing performance?

Task #5: Dropout

Investigate the effects of dropout by comparing:

- Model A: Dropout rate = 0.0
- Model B: Dropout rate = 0.2
- Model C: Dropout rate = 0.5

Apply dropout after the hidden layers, keeping the rest of the model configuration consistent.

Example:

from tensorflow.keras.layers import Dense, Dropout
model_layers = [Dense(32, activation="relu"), Dropout(0.2), Dense(16, activation="relu"), Dropout(0.2), Dense(1, activation="sigmoid")]

Questions:

1. How does dropout work during training?
2. Why can randomly deactivating neurons reduce overfitting?
3. Why is dropout disabled during inference?
4. Does a higher dropout rate always improve model performance?

Task #6: Overall Model Performance and Evaluation

Develop an improved MLP using the findings from the previous tasks. Select an appropriate combination of optimiser, hyperparameters, L2 regularisation, and dropout. Evaluate the final model using:

- Accuracy, Precision, Recall, F1-score
- Confusion matrix
- Training and testing loss curves
- Training time

Compare the final model with the baseline and discuss the effects of the applied techniques.

Because loan decisions can have financial consequences, you must also discuss false approvals, false rejections, potential bias, and why model performance alone does not establish that a model is suitable for real lending decisions.

Questions:

Write a technical discussion addressing:

1. Which model configuration performed best, and why?
2. Which technique most effectively controlled overfitting?
3. Did any configuration perform well on training data but poorly on validation data?
4. How reliable is the selected model on unseen applications?
5. What are the limitations of the dataset and the model?

Document the Python library versions, random seeds, dataset preparation steps, and experimental settings.

Task #7: Neural Network Model Deployment

Deploy the best-performing Multilayer Perceptron (MLP) model developed in the loan approval prediction assignment as a simple application that accepts applicant information and predicts whether a loan should be approved or rejected.

1. Save the trained model
   - Save the best-performing MLP model in .pkl / .joblib format to ensure that the saved model can be loaded without retraining.
2. Develop a prediction interface
   - Build a simple web interface using Streamlit / Flask / Tkinter .
   - Allow users to enter applicant information, such as annual income, loan amount, CIBIL score, loan term, and asset values.
   - Confirm that the application handles invalid inputs appropriately.
   - Display the predicted class and predicted probability.
   - Confirm that the application handles invalid inputs appropriately.

Questions:

1. How can a developer ensure that the deployed model produces the same predictions as the original model?
2. What is prediction latency, and why is it important when deploying a loan approval prediction system?
3. How would you handle missing, invalid, or out-of-range applicant information?
4. Why should the model's predicted probability not automatically be interpreted as a guarantee of loan approval?
5. What risks could arise if the model performs differently across demographic or socioeconomic groups?
6. How could model drift be detected after deployment, and when might retraining be necessary?
7. Should the deployed system automatically approve or reject loans, or should a human decision-maker review its recommendations? Justify your answer.

Submission Materials and Requirements:

Students must submit:

- Python source file (.py) that runs on the supplied dataset.
- A report describing the methodology and findings.
- Tables comparing optimiser, hyperparameter, regularisation, and dropout experiments.
- Learning curve graphs and confusion matrices.
- A discussion of overfitting, generalisation, limitations, and the implications of prediction errors in loan approval.

Also, you are required to prepare **FIVE (5)** presentation slides for oral presentation purposes.

Policy:

All grading of deliverables will be based on the standards indicated for each deliverable. Deliverables may not be turned in late, and no cheating! For this class, cheating will include plagiarism (using the writings of another without proper citation), copying of another (either current or past student's work), working with another on individually assigned work, or in any other way presenting as one's work that which is not entirely one's work. The occurrence of plagiarism will result in removal from the course with a failing grade.